

Practical Assignment 3

Volumetric Maximum Intensity Projection (MIP) with CUDA Programming

1 - Introduction

The **Stanford Bunny** (Figure 1) is perhaps the most iconic test model in the history of computer graphics. Originally scanned in 1994, it has served as the baseline for algorithms ranging from mesh simplification to ray tracing. In this assignment, we move beyond the surface mesh and work with the **Stanford Bunny CT Scan**, a volumetric dataset that captures the internal density of the model, not just its “skin.”

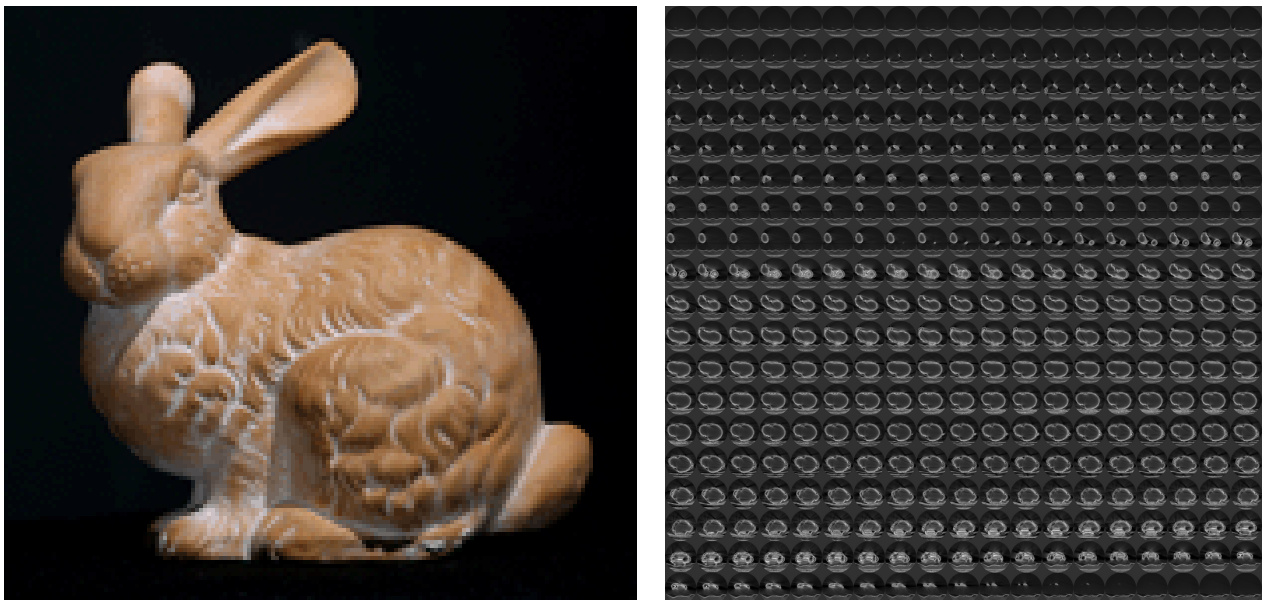


Figure 1: The Standford Bunny (left) and its CT scan (right)

1.1 - The Volumetric Dataset

Unlike a 3D mesh composed of triangles, a **CT (Computed Tomography) scan** consists of a series of 2D cross-sectional “slices” (Figure 1). For this project, you are provided with **361 slices**, each a 512×512 grayscale image. When stacked together, these slices form a 3D grid of volume elements, or **voxels**.

This volume is represented in memory as a contiguous 1D array of `uint16_t` values, where each value represents the density at a specific point (x, y, z) . The total volume dimensions are $512 \times 512 \times 361$.

1.2 - The Processing Pipeline

To transform this raw “cloud” of density data into a high-quality 2D raster image, your program must implement a three-stage pipeline. Each stage presents a unique parallelization challenge in CUDA.

- 1) **Thresholding (Point Operation):** Raw CT scans often contain “air noise”, low-intensity values captured in the space surrounding the object. You will implement a threshold function that sets any voxel below a specific intensity to zero. This “cleans” the volume and ensures that the final projection is focused solely on the bunny.

-
- 2) **3D Gaussian Blur (Stencil Operation):** To prevent single high-intensity noise pixels from dominating the final image, you will apply a 3D Gaussian blur. This stage requires a spatial convolution where each voxel's value is recalculated based on a weighted average of its $3 \times 3 \times 3$ neighborhood.
 - 3) **Maximum Intensity Projection (MIP):** MIP is a volume rendering method used to visualize 3D data on a 2D plane. It simulates firing parallel rays through the volume. For each pixel on the screen, the algorithm finds the maximum voxel intensity encountered along the corresponding ray.

By implementing this on the GPU, you will take a process that takes seconds on a CPU and accelerate it to real-time speeds, allowing for interactive rotation and visualization of the bunny's internal structure.

2 - Requirements

Each group is assigned the task of porting the sequential volume processing pipeline to the GPU using the CUDA framework.

2.1 - Base Code and Environment

A reference implementation is provided in C/C++, located in the directory `src/bunnyMIP`. This code contains the sequential logic for loading the volume, thresholding, blurring, and projecting the data.

Your implementation work must be contained within:

- `src/device-bunnyMIP.cu`

You are expected to write the kernel definitions, handle memory allocation on the device (VRAM), and manage the data transfer between the Host (CPU) and Device (GPU).

2.2 - Validation and Correctness

To verify the success of your implementation, your application must perform a validation check. The raster image generated by your CUDA kernels must be compared against the “Gold Standard” image generated by the host. The device-calculated MIP must not differ significantly from the host-calculated version.

2.3 - Performance Evaluation: Speedup Analysis

A portion of your final report must be dedicated to a quantitative analysis of your GPU implementation. You must demonstrate that your parallelization efforts resulted in a measurable performance gain over the sequential reference.

2.4 - Command Line Interface (CLI)

Your application must be dynamic. It must parse and accept the following input parameters to influence the final rendering:

- `--threshold`: Defines the intensity floor. Any voxel $v < \text{threshold}$ must be set to 0.
- `--sigma`: Defines the standard deviation (σ) used to calculate the Gaussian weights.
- `--roll`, `--pitch`, `--yaw`: Defines the rotation of the volume in degrees.

These parameters must be used to update the kernels at runtime. Specifically, the rotation angles should be used to construct a 3×3 rotation matrix R , which is then utilized by the MIP kernel for backward mapping.

2.5 - Computational Environment

All development and performance profiling for this project must be conducted on the **banana.ua.pt** server. This server is equipped with the necessary NVIDIA hardware and CUDA toolkit to support your volumetric pipeline. The login credentials for each group are **cleNN**, where **NN** is the group number (e.g., **cle01**, **cle03**, **cle14**).

The password will be provided to each group upon request in class or by email.

2.5.1 - Shared Resource Etiquette

The **banana** server is a **shared resource** used by multiple student groups simultaneously. To ensure a stable and fair environment for everyone, please adhere to the following rules:

- **Responsible Usage:** Be mindful of your VRAM and CPU usage. Avoid leaving large memory allocations active if you are not actively debugging.
- **No VSCode SSH:** You are prohibited from using the **VSCode Remote-SSH extension**.
 - **The Reason:** This extension is notoriously resource-heavy. It spawns multiple background Node.js “zombie” processes that consume significant RAM and CPU cycles on the host, even after you disconnect. This can drastically degrade the performance of the server for other users.
- **Alternative Tools:** Please use lightweight terminal-based editors (such as **vim** or **nano**) or develop locally and transfer your files using **scp** or **rsync**. If you prefer a GUI, use a local editor and synchronize your code via **Git** or **SFTP**.

Note: Failure to comply with server etiquette, especially regarding the use of prohibited extensions, may result in your processes being terminated without warning or, in extreme cases, the suspension of your server access. Treat the hardware with respect; it’s the only bunny-processing power we’ve got!

3 - Evaluation

The evaluation of this project is divided into two main components: the technical implementation (80%) and a final presentation (20%). **The presentation is mandatory.** All groups must maintain their source code in the **banana.ua.pt** group home directory, which must include a **README.md** file documenting the architectural decisions and concurrency strategies employed.

3.1 - Presentation (20%)

Following the code submission, each group will perform a presentation. The total duration is strictly limited to **15 minutes**, structured as follows:

- **Presentation (5 min):** A concise overview of the implemented CUDA kernels.
- **Demo & Q&A (10 min):** A live demonstration of the software, during which the instructor may ask technical questions regarding the implementation.

3.2 - Technical Implementation (80%)

The submitted project will be evaluated based on the following weighted criteria, focusing on technical correctness, software engineering standards, and documentation quality.

Evaluation Criteria

- **Functionality (55%)**
 - All core features are working correctly

- **Code Quality (15%)**
 - Proper use of CUDA programming (10%)
 - Code readability and comments (5%)
- **Documentation (15%)**
 - Complete and clear `README.md` (10%)
 - Instructions on how to compile and run the application.
 - Explanation of the developed CUDA kernels.
 - A speedup analysis of your code.
 - Other information considered relevant by the group.
 - Technical documentation quality (5%)
- **Innovation (15%)**
 - Implementation of the following additional features
 - Add the capability to calculate N MIP raster images with different rotations in the device with one CUDA kernel call.
 - Generate a video of the outputed N raster images using an external tool such as `ffmpeg`.

3.3 - Academic Integrity Guidelines

This assignment is a **collaborative group effort**. While you are encouraged to discuss general concepts with peers from other teams, your final codebase must be the **unique, original creation** of your specific group.

Usage of AI & External Tools

- **Permitted:** Using AI assistants to troubleshoot bugs or clarify complex theoretical concepts.
- **Prohibited:** Using AI to generate entire solutions or major functional blocks of code.
- **Citations:** You must explicitly credit any external code snippets or concepts sourced from the web or elsewhere.

Standards & Consequences

- **Formatting:** All references and citations must strictly adhere to **IEEE templates**.
- **Plagiarism:** There is a zero-tolerance policy for academic dishonesty.
- **Penalty:** Any group found in violation of these rules will face **immediate disqualification** of their work.

4 - Deliverable

Your source code should be located in the `banana.ua.pt` group home directory by the delivery deadline. Ensure you meet the deadline, as delays will not be accepted.

Ensure that your directory includes:

- **All source files.**
- A `README.md` file with setup instructions, usage details, and a summary of your approach.
- Performance analysis results, including execution time comparisons and speedup calculations.

4.1 - Deadline and Protocol

The submission window closes on **June 1 at 9:00**.